ORDERDESK · RETAIL PARTNER PORTAL

Complete Workflow Guide

A practical guide to running the app, placing a wholesale order, following fulfillment, and understanding how the frontend and backend work together.

At a glance

OrderDesk is a React and Vite ordering portal backed by a lightweight Node.js API. The catalogue and initial order history are demo data. The portal lets a retail partner build a case-quantity order, submit it for validation, and review the resulting order and invoice details.

1 · Browse
Search and filter the wholesale catalogue.

›

2 · Build cart
Add products in case-size multiples.

›

3 · Checkout
Review the order and confirm it.

›

4 · Track
Review order status and invoice details.

Start the application

Install Node.js and npm. From the repository root, open two terminals:

TERMINAL 1 · BACKEND

```
cd Project2\backend
npm run dev
```

API listens at 127.0.0.1:3001. No backend package installation is required.

TERMINAL 2 · FRONTEND

```
cd Project2\frontend
npm install
npm run dev
```

Open the local Vite URL printed in the terminal. Vite forwards /api calls to the backend.

Startup data flow

When the portal opens, the frontend requests `GET /api/products` and `GET /api/orders`. If the API cannot be reached, checkout stays unavailable and the page offers a retry action. The backend loads saved orders from `backend/data/orders.json`; if no file exists, it uses the sample orders in `backend/seed.js`.

PART 1 · RETAIL PARTNER JOURNEY

From catalogue to checkout

Step	What the retailer does	What the portal does
Browse	Search by product, brand, or category. Choose All products, Kitchen, or Home & living.	Filters the loaded catalogue in the browser; it does not make a new API request for each search.
Add products	Use Add or the plus/minus controls to choose case quantities.	Starts at each product's MOQ, adjusts in MOQ increments, caps at displayed stock, and saves the cart in browser local storage.
Review cart	Check quantities, subtotal, discount, shipping, and estimated total.	Includes shipping in the displayed total. Shows progress toward the discount threshold.
Checkout	Open checkout, review items and delivery estimate, then choose Place order.	Sends product IDs and quantities to `POST /api/orders`. No payment is collected.
Confirmation	Continue to My orders after successful submission.	Shows the new order, clears the cart, and displays a success notice.

Ordering rules

MINIMUM

₹5,000 subtotal

The backend rejects an order below the minimum value.

CASE QUANTITIES

MOQ multiples only

Every quantity must be a whole number and a multiple of that product's MOQ.

STOCK CHECK

Quantity cannot exceed listed stock

The backend checks the requested quantity against the sample catalogue stock value.

DISCOUNT

5% at ₹50,000+

Discount is calculated from subtotal and rounded to the nearest rupee.

Delivery estimate: the order record uses dates five and seven days from placement. The checkout screen describes this as an estimated 5–7 business days; it is a prototype estimate, not live carrier tracking.

Cart persistence

The cart is stored in this browser under `orderdesk-cart-v1`. It survives refreshes in the same browser, but is not an account-synced or server-side cart.

PART 2 · BACKEND ORDER PROCESSING

What happens when an order is submitted

1. The frontend sends a JSON body containing a `lines` array. Each line contains a product `id` and integer `quantity`.
2. The backend checks that the order is non-empty, contains no more lines than catalogue products, and does not repeat a product.
3. It checks that each product exists, quantities are whole numbers and MOQ multiples, stock is not exceeded, and subtotal reaches ₹5,000.
4. The backend calculates the 5% discount when eligible, assigns an order ID and invoice reference, and sets the initial status to Processing.
5. The new order is written before the API responds. Writes are serialized and saved using a temporary file followed by a rename.
6. On success, the frontend adds the order to the current history, clears browser cart storage, and opens My orders. On failure, it displays an error and keeps the cart contents.

API reference

Method and path	Purpose	Successful response
`GET /api/health`	Backend health check.	`{ "status": "ok" }`
`GET /api/products`	Read the sample catalogue.	`{ "products": [...] }`
`GET /api/orders`	Read saved or seeded orders.	`{ "orders": [...] }`
`POST /api/orders`	Validate, create, and persist an order.	HTTP 201 with `{ "order": ... }`

Order record

Each record contains an order ID (for example, `OD-3000`), placement date, status, delivery estimate, subtotal, discount, total, product lines, and invoice reference (for example, `INV-3000`).

Persistence and sample data

Submitted orders are saved in `Project2/backend/data/orders.json`. The data directory is excluded from Git. If the file is absent, the backend serves the four sample orders from `seed.js`; it creates the data file when the first new order is successfully saved.

> **Errors:** invalid JSON or order data returns a client error with an explanation. Unexpected storage errors are logged by the backend and returned as a server error.

PART 3 · ORDERS, INVOICES, AND SUPPORT

Follow an order and review invoice details

MY ORDERS

History and status

Review order date, Processing / In transit / Delivered status, estimated delivery, total, and order details. Filter the list by status. The summary cards count open orders, delivered orders, and total order value.

INVOICES

Invoice list and preview

Invoice rows use the same order records. Selecting View opens an in-app invoice details preview with product lines, quantities, values, and totals.

> **Important:** the invoice View/Download-labeled actions open the in-app preview. They do not generate or download an invoice PDF. Payment processing is not implemented.

Connection troubleshooting

Symptom	What to check
Backend connection issue in the portal	Confirm the backend terminal is running and listening on 127.0.0.1:3001. Then select Retry connection.
Checkout cannot be opened	The frontend must successfully load both catalogue and order history from the API.
Order rejected	Check the ₹5,000 minimum, product IDs, MOQ multiples, whole-number quantities, and available stock.
Orders reset after removing local data	Order history is stored in `backend/data/orders.json`. Back up this file before deleting local project data.

Prototype boundaries

- Catalogue, store profile, account credit, and starter orders are sample/demo data.
- There is no sign-in, authorization, payment processing, or real shipment integration.
- Stock values are checked during order submission, but inventory is not decremented or reserved. Concurrent orders are not protected by a production inventory system.
- Order storage is a local JSON file intended for this prototype, not a production multi-user database.

Build the frontend

From `Project2\frontend`, run `npm run build`. Vite writes the production frontend to `frontend/dist`. The included backend is an API only; a static web server or hosting platform must serve the built frontend.